

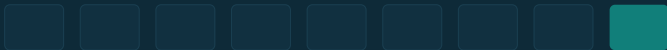


STREAM PROCESSING AVEC APACHE KAFKA

Sérialisation & Schema Registry

Avro · format de trame · évolution compatible des schémas

ESGI · 5^e année · Data Engineering · 2025–2026



SÉANCE 7

Objectifs

- **Comprendre le rôle d'un schéma**

Un message est une suite d'octets : producteur et consommateur ont besoin d'un contrat.

- **Comparer les formats**

JSON, Avro, Protobuf — compacité, schéma, évolution.

- **Utiliser un Schema Registry**

Format de trame (magic byte + ID), sujets, `KafkaAvroSerializer`.

- **Faire évoluer un schéma sans rien casser**

Compatibilité `BACKWARD` / `FORWARD` / `FULL` et ordre de mise à jour.

SÉANCE 7

Au programme

01

Sérialisation & formats

JSON · Avro · Protobuf

02

Schema Registry

trame, sujets, ID

03

Évolution & compatibilité

backward, forward

04

Atelier Avro

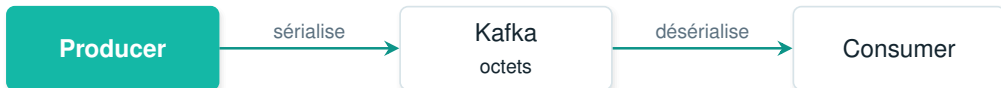
produire, faire évoluer

01

- PARTIE 1

Sérialisation & formats

Un message n'est qu'une suite d'octets



■ Sans contrat

Le consommateur *devine* la structure. Un champ renommé ou retiré casse silencieusement.

■ Avec un schéma

Le format est explicite, validé, et peut **évoluer** de façon contrôlée.

PARTIE 1 · COMPARER

JSON, Avro, Protobuf

Format	Taille	Schéma	Évolution	Lisible
JSON	grande	implicite	fragile	oui
Avro	compacte	requis (.avsc)	excellente	non
Protobuf	compacte	requis (.proto)	bonne	non

Avro est le format de référence de Kafka : binaire compact, schéma séparé, évolution claire.

Un schéma .avsc

`order-v1.avsc` - un enregistrement Avro

```
{
  "type": "record",
  "name": "Order",
  "namespace": "fr.esgi.kafka",
  "fields": [
    {"name": "orderId", "type": "string"},
    {"name": "amount", "type": "double"},
    {"name": "currency", "type": ["null", "string"], "default": null}
  ]
}
```

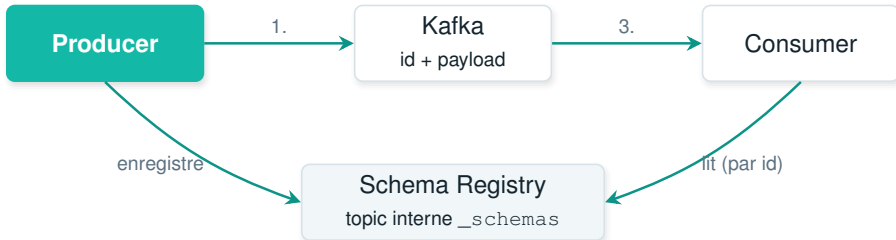
Un champ **optionnel** = union ["null", "string"] avec "default": null (le null en premier).

02

- PARTIE 2

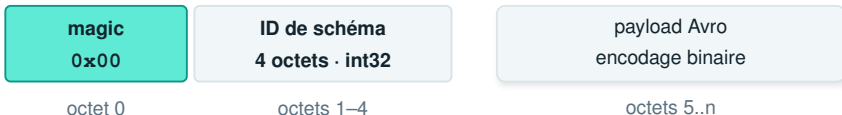
Schema Registry

Un registre central des schémas



- Service **REST** qui stocke et versionne les schémas, attribue un **ID** à chacun, et applique les règles de compatibilité.
- **Composant Confluent**, pas Apache Kafka : il se branche en client sur le cluster (image `cp-schema-registry`).

Magic byte + ID de schéma + payload



Cote producteur : on remplace juste le serialiseur

```
props.put("value.serializer", KafkaAvroSerializer.class.getName());  
props.put("schema.registry.url", "http://localhost:8081");
```

Sujets par défaut : <topic>-value et <topic>-key (auto-enregistrés au premier envoi).

Produire en Avro, inspecter le registre

Produire un message Avro, puis interroger le registre REST

```
# 1) Produire en Avro (producteur du kit, order-v1.avsc)
mvn -q compile exec:java \
  -Dexec.mainClass=fr.esgi.kafka.tp7.AvroOrderProducer
# 2) Le registre a cree le sujet et stocke le schema
curl -s localhost:8081/subjects
curl -s localhost:8081/subjects/orders-value/versions/1
```

■ Sujet créé

/subjects renvoie orders-value ; la v1 contient le schéma.

■ Octets bruts

Sans désérialiseur Avro, un kafka-console-consumer n'affiche que des octets.

03

- PARTIE 3

Évolution & compatibilité

Quels changements sont autorisés ?

Mode	Lecture	Changements permis	Mettre à jour
BACKWARD (défaut)	nouveau lit l'ancien	supprimer ; ajouter avec défaut	consommateurs
FORWARD	ancien lit le nouveau	ajouter ; supprimer (avec défaut)	producteurs
FULL	les deux	uniquement champs avec défaut	n'importe
NONE	—	tout (aucun contrôle)	à éviter

Variantes *transitives* : elles vérifient *toutes* les versions, pas seulement la dernière.

Ajouter un champ, sans casser

v1 -> v2 : on ajoute "status" AVEC un default (compatible BACKWARD)

```
// order-v1.avsc
{"name": "orderId", "type": "string"},
{"name": "amount", "type": "double"}

// order-v2.avsc (ajout retro-compatible)
{"name": "orderId", "type": "string"},
{"name": "amount", "type": "double"},
{"name": "status", "type": "string", "default": "NEW"}
```

En BACKWARD, un consommateur v2 lit un message v1 : le champ `status` prend la valeur "NEW".

Compatible vs rejetée

Enregistrer une v2 compatible, puis une modification incompatible

```
# Produire en v2 (status a default) : auto-enregistre si compatible
mvn -q compile exec:java \
  -Dexec.mainClass=fr.esgi.kafka.tp7.AvroOrderProducer -Dexec.args="v2"
# Le sujet compte maintenant deux versions
curl -s localhost:8081/subjects/orders-value/versions      # -> [1,2]
```

À observer : la v2 renvoie `{"id":N}`. Un champ **requis sans défaut**, ou un changement de type, est refusé avec 409 `Conflict` : le registre protège les consommateurs.

04

- PARTIE 4

Atelier — produire et faire évoluer de l'Avro

Cluster + Schema Registry

Demarrer les brokers et le registre, verifier qu il repond

```
docker compose up -d                                # 3 brokers + schema-registry

kafka-topics.sh --bootstrap-server localhost:29092 \
  --create --topic orders --partitions 3 --replication-factor 3

# Le registre repond sur le port 8081 (liste vide au depart)
curl -s localhost:8081/subjects
```

■ Registre

cp-schema-registry branché sur les brokers, REST sur localhost:8081.

■ Projet

Maven + kafka-avro-serializer (dépôt Confluent requis dans le pom).

Produire, inspecter, faire évoluer

- 1 Compléter `AvroOrderProducer : KafkaAvroSerializer + schema.registry.url`, charger `order-v1.avsc`, construire un `GenericRecord`.
- 2 Produire quelques commandes, puis `curl` les sujets et versions.
- 3 Compléter `AvroOrderConsumer` et lire les commandes décodées.
- 4 Passer au schéma `order-v2.avsc` (champ `status` avec défaut), produire en v2.
- 5 Re-consommer : les messages v1 et v2 se lisent (défaut appliqué).

Le registre raconte l'évolution

Suivre les versions du sujet

```
# Toutes les versions enregistrées pour la valeur du topic
curl -s localhost:8081/subjects/orders-value/versions      # ex: [1,2]

# Le détail d une version donnée
curl -s localhost:8081/subjects/orders-value/versions/2

# Le mode de compatibilité courant (BACKWARD par défaut)
curl -s localhost:8081/config/orders-value
```

■ Deux versions

[1, 2] : v1 et v2 coexistent, chacune avec son ID.

■ Lecture mixte

Le consommateur v2 lit les anciens messages grâce au défaut.

Pour aller plus loin

Objectif — éprouver les règles de compatibilité et toucher du doigt `SpecificRecord`.

■ 1. Casser

Tenter un champ requis sans défaut $\Rightarrow$ 409. Le corriger en ajoutant un défaut.

■ 2. FORWARD

Passer le sujet en `FORWARD` et observer quels changements deviennent permis.

■ 3. `SpecificRecord`

Générer une classe Java typée depuis le `.avsc` et l'utiliser.

Au-delà d'Avro

- **Protobuf** et **JSON Schema** sont gérés par le même registre, avec leurs propres règles de compatibilité et leurs sérialiseurs dédiés.
- **Alternatives open-source** : `Apicurio Registry` (Red Hat) expose une API compatible et un mode multi-format.
- **Data contracts** : au-delà du schéma, on attache règles de validation, métadonnées et qualité de la donnée.

■ Le format

Avro, Protobuf ou JSON Schema : un choix par flux, pas global.

■ Le contrat

Le registre devient le point de gouvernance des données.

SÉANCE 7

À retenir

- ✓ Un message est une suite d'octets : le schéma est le contrat producteur–consommateur.
- ✓ Avro : binaire compact, schéma `.avsc` séparé, évolution maîtrisée.
- ✓ Le Schema Registry stocke les schémas et applique la compatibilité ; trame = magic byte + ID + payload.
- ✓ BACKWARD (défaut) : ajouter un champ **avec défaut**, mettre à jour les consommateurs d'abord.
- ✓ Sujets `<topic>-value / <topic>-key` ; un changement incompatible est refusé.



LA SUITE — SÉANCE 8

Kafka Connect

Intégrer des sources et des sinks externes sans code, avec connecteurs et convertisseurs (dont Avro).